Introduction to Causal Inference for Software Engineering

A 90-minute hands-on tutorial

Julien Siebert · Julian Frattini · Hans-Martin Heyn · Roberto Pietrantuono

FSE 2026 · Montréal · 5–9 July 2026

Welcome to a tutorial on causal inference in SE

- This tutorial is for researchers and practitioners who like to explore what causal inferences can offer for Software Engineering.
- We do not require prior knowledge in statistics or mathematics. We will work mostly with graphical methods for causal inference.
- Lecture materials, exercises, and reading suggestions are available at: https://github.com/martinheyn/CI4SE_Tutorial

PART 1 · OVERVIEW

Motivation and modeling, grounded in SE

1. **Motivation** — Why SE questions are causal, not just associational
2. **Modeling** — Causal graphs, confounders, and Pearl's process
3. **Examples** — Worked recipes for testing and fault localization
4. **The road ahead** — Where causal software engineering is heading

How machines have supported software engineers

1

Scaffolding

Organizing & structuring work

IDEs · version control · linting

2

Support automation

Formalized methods for tasks

Coverage testing · static analysis

3

Support decision

Learn from data; predict

Estimation · fault prediction · RCA

4

Support reasoning

Explain, hypothesize, plan

What-if · counterfactuals · RCA

MOTIVATION · BEYOND CORRELATION

Example 1: Same improvement — but what caused it?

Scenario A microservice team ships a new client-side retry policy to cut tail latency. Within an hour, latency drops.

3

Correlation view

AIOps attributes the drop to the deploy: “the change fixed latency.”

Plan: replicate the pattern on other services.

⚠ But autoscaling increased replicas in a downstream dependency and a traffic shift moved requests away from a “hot” region: both can reduce queueing delays. So is the improvement due to the new retry policy, to autoscaling, traffic shift or their interaction?
When repeating under different conditions, latency regresses → incident.

4

Causal view (CSE)

Estimate the effect of do(X) net of confounders Z, with uncertainty — then act on it:

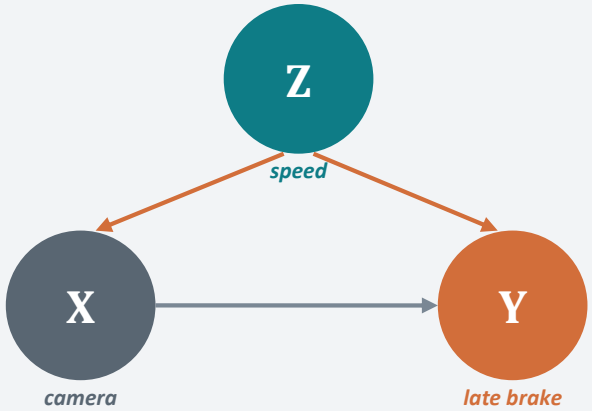
→ Strong, robust effect: proceed / widen the canary
→ Real but modest: proceed, but stop tuning
→ Tied to one factor: target the likely cause, apply that fix elsewhere

Example 2: Blaming the wrong cause

A shuttle **brakes too late**. A correlation-based tool finds late braking (Y) co-occurs with high-res camera mode (X) — and blames the camera.

But a confounder drives both: **higher speed (Z)** triggers the camera switch and shrinks the braking margin.

A causal model answers what-if questions:
"What if we adjust the speed threshold?"



Adjust for Z to remove the spurious X–Y link.

MOTIVATION

Software engineering is increasingly data-driven

Machine learning and LLMs now support nearly every SE activity — **prediction, generation, recommendation** across the lifecycle. This data-driven paradigm scales one human ability: **pattern recognition**.

What it does well

Learns associations from large datasets; automates and amplifies inductive reasoning.

Where it stops

Stays at rung 1 — association. It predicts from what it has seen, not what would happen if we act.

MOTIVATION

The questions SE asks are causal, not associational

Observing

$P(Y \mid X = x)$

"What outcome when we happen to see X = x?"

Good for prediction. Blind to confounding.

Intervening

$P(Y \mid do(X = x))$

"What outcome if we actively set X = x?"

Behind testing, debugging, tuning, policy change.

Most worthwhile SE questions are causal — but answers can't come from data alone (Pearl).

MOTIVATION

Climbing the ladder of causation — in SE terms

1 **Association** · *Seeing*

"What if I see…?" Past tests & observations correlate with failures.

2 **Intervention** · *Doing*

"What if I do…?" Set an input and predict the effect — what-if analysis, test selection.

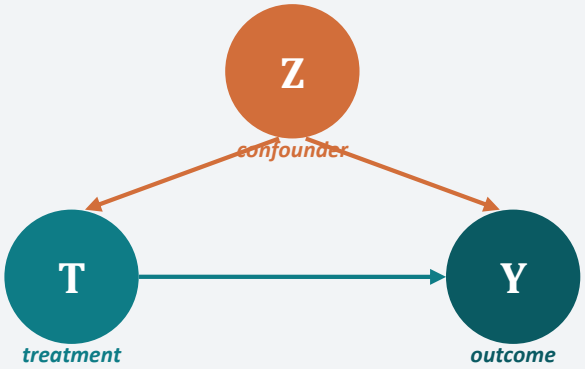
3 **Counterfactual** · *Imagining*

"What if I had done…?" Would the failure have occurred otherwise? Root-cause & debugging.

A causal graph makes assumptions explicit

Nodes = variables. **Directed links** = direct causal effects. A DAG encodes hypotheses we can reason about and test.

- **Confounder** — common cause of T and Y — induces spurious correlation; adjust for it
- **Mediator** — lies on T → M → Y — carries part of the effect
- **Collider** — common effect — conditioning on it opens a spurious path



Z biases the T → Y estimate unless we adjust for it.

The process: from assumptions to validated effects

1 — Model
State assumptions as a causal graph

→

2 — Identify
Can the effect be estimated from data?

→

3 — Estimate
Compute the effect (regression, ML, …)

→

4 — Refute
Try to falsify the estimate

The graph is a collaborative reasoning partner — inspectable, contestable, and refinable by engineers.

EXAMPLES FROM SE RESEARCH

Three domains where causal graphs are taking hold

Testing

Choose inputs that expose failures — what-if reasoning over a model of the system under test.

Fault localization

Trace a failure to its cause; separate the culprit from mere correlates.

Requirements engineering

Causal models as requirements engineering activity

For each: first the big picture, then a concrete, repeatable recipe.

EXAMPLES · TESTING — WHERE WE ARE

Testing is naturally causal

1. **Guessing → Tester intuition/expertise**
   "what might fail?"

2. **Correlation → ML on past tests/data**
   "what correlates with failure?"

3. **Causation → Interventions**
   "what input causes it to fail?"

Choosing a test input implicitly predicts an outcome under a hypothetical condition — **a what-if question.**

Test generation is **planning**, not prediction: what output if we submitted this input? Causal models let testers reason, not guess.

EXAMPLES · TESTING — EXAMPLE 1

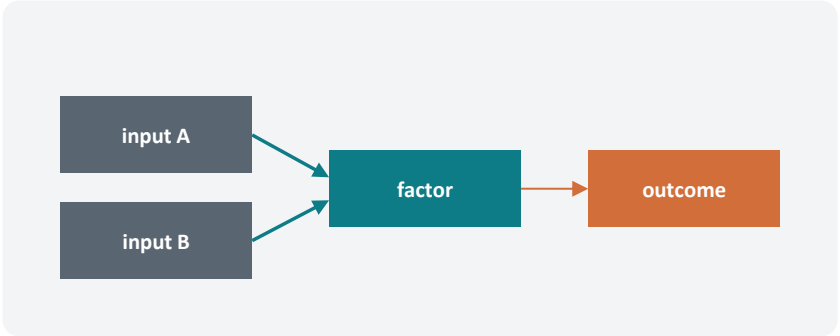
A recipe for Causal Test Generation

Generating tests by intervening on a causal model

Goal. Pick inputs most likely to expose a failure instead of sampling blindly.

1. **Model** — over inputs, internal factors, and the outcome
2. **Learn** — fit from a few runs or domain knowledge
3. **Intervene** — query do(input = x) to predict the outcome
4. **Select** — run only configs predicted to breach the requirement

Reusable wherever inputs map to a measurable requirement.



$P(Y \mid X=x)$ vs $P(Y \mid do(X=x))$

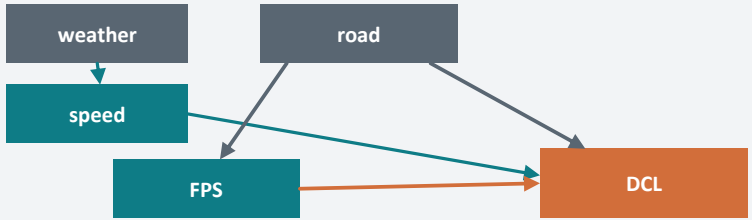
Observing vs. setting — only do(·) reveals which inputs cause the failure.

EXAMPLE 1 · IN PRACTICE

Concretely: testing a self-driving car in a simulator

Setup. A simulator with ~16 scene inputs and a safety requirement: **staying in lane (DCL)**.

Learned causal model (excerpt)



A counterfactual question

Observed: rain, curved, FPS=30 → safe

Counterfactual: had FPS been 20?

naïve : P(DCL>thr | FPS=20) = 0.85

causal: P(DCL>thr | do(FPS=20)) = 0.55

Road confounds FPS → DCL — adjusting for it makes do(FPS) ≠ P(DCL | FPS). That 0.85→0.55 gap is the causal payoff.

Concrete example after: Giamattei et al., Causality-driven Testing of Autonomous Driving Systems (ACM TOSEM 33(3), 2024).

The same recipe for non-functional testing

Testing is also about **planning what-if scenarios**.

Keep the Example 1 recipe; just **change the outcome variable**:

"Which workloads push latency past the threshold?"

Functional
outcome = requirement violated

Performance
outcome = latency past threshold

Fairness, energy…
outcome = any quality metric

EXAMPLE 2 · IN PRACTICE

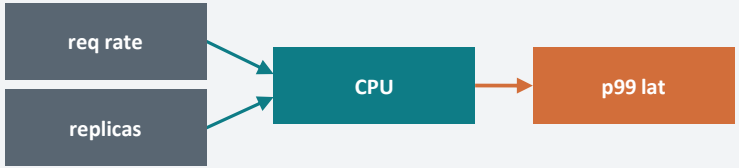
Concretely: which workload breaches the SLO?

Setup. A microservice under load. The model links workload & resources to **p99 latency**.

The what-if query

```
P(latency > SLO | do(rate=high, replicas=2))
```

Causal model over workload & resources



intervention	p99	verdict
`rate ↑, replicas = 2`	**1.8× SLO**	breach
`rate ↑, replicas = 4`	**0.9× SLO**	ok
`async call`	**0.7× SLO**	ok

Concrete example after: Giamattei et al., Identifying Performance Issues in Microservices (AST 2024).

Fault localization: the leading application

~50%

of surveyed papers

*Causal reasoning is seen as a way to **explain why** an event happened.*

It maps onto the chain from a fault to a failure. Two flavours dominate:

- **Debugging** — Locate the faulty statement and the fix — statistical FL, program repair.
- **Root cause analysis** — Find the culprit component in a running system — esp. microservices & cloud.

True causal inference — e.g. counterfactual RCA — is still uncommon; most work learns the structure, then ranks.

*References to Julien and mine surveys

Tracing a failure to its true cause

In a system of interacting components, a failure **propagates**. A causal model separates the real culprit from components that merely *correlate* with the symptom.

Propagated — An inner failure cascades; the edge component gets blamed.

Masked — A symptom is hidden downstream; black-box checks pass.

The counterfactual query

$$P(fail_i \mid do(comp_k = fail))$$

"If component k fails, which others fail too?"

Interventions trace the chain **without injecting real faults**.

comp k → comp j → symptom

Inspired by: Johnson, Brun & Meliou, Causal testing: understanding defects' root causes (ICSE 2020).

EXAMPLE 3 · IN PRACTICE

Concretely: tracing a failure across services

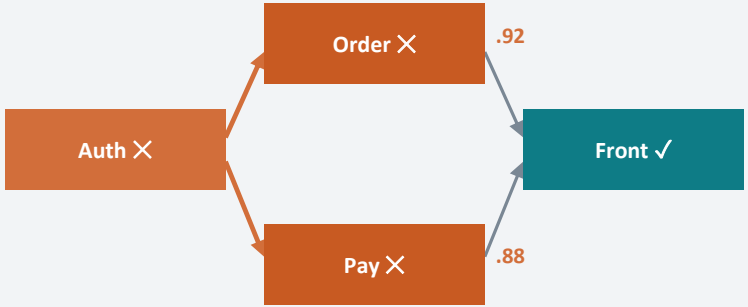
Setup. A benchmark of ~40 services. A failure in an inner service shows up at the edge.

The intervention

```
P(fail(Pay), fail(Order) | do(Auth = fail))
```

Synthetic result: *Auth=fail propagates to Order (0.92) & Payment (0.88); front-end masked.*

do(Auth = fail) — propagation traced



✕ fails under do(·) ✓ stays up (failure hidden)

Concrete example after microservice RCA literature; cf. Wu et al. (2021). (TO MAKE EXPLICIT)

A repeatable recipe for causal RCA

Whatever the system, the same three moves turn telemetry into an explainable root cause.

1 · Build the graph

Components & runtime variables as nodes; edges from architecture or causal discovery.

→

2 · Reason on it

Trace propagation with do(comp = fail); ask counterfactuals about alternatives.

→

3 · Adjust & explain

Control for confounders; return an inspectable causal chain.

Why it transfers: independent of the domain — microservices, cloud, cyber-physical, ML pipelines. Swap variables, keep the reasoning.

How a practitioner actually uses causal inference

The examples so far were end results. Underneath them is **one repeatable two-step process**: build a causal model, then query it — with humans in the loop throughout.

1

Step 1 — Learning

Construct a causal model — from a specification, from data, or both

→

2

Step 2 — Inference

Query the model — what-if (proactive) & counterfactual (retroactive)

The causal model is a **collaborative reasoning partner**, not a black-box oracle — engineers can inspect, challenge, and override it.

THE PROCESS · STEP 1

Building the causal model (learning)

Three sources, often combined — the model encodes **what we know and what we measure**.

Specification

Experts declare cause-effect relations - which links must hold, which are forbidden. An inspectable record of assumptions; LLMs can suggest variables.

Domain expertise, assumptions-based

Discovery

Causal-discovery algorithms extract structure from data - observational or experimental, structured or text. Engineers validate & revise.

Observations/data, evidene-based

Fitting

However the structure is obtained (via Specification, Discovery or Hybrid) data parameterize it – e.g., structural equations (SCM) or conditional distributions (CBN).

Validation: if observations contradict a specified relation, revisit whether the assumption is wrong - or the data simply insufficient.

Pietrantuono, Giamattei & Russo, Reasoning Beyond Prediction: From Data-Driven to Causal Software Engineering, CACM (to appear in Dec 2026) – https://arxiv...

THE PROCESS · STEP 2

Querying the model (inference)

With a model in hand, engineers ask two kinds of question - and can **refute** the answers.

Proactive - what-if

`do(X = x)`: predict the effect of an action before taking it.

Design exploration, test generation, performance tuning.

Retroactive - counterfactual

would the failure have occurred had X been different?

Root-cause analysis, debugging, corrective maintenance.

Estimate with uncertainty, then refute: effect estimates come with confidence intervals and are checked by refutation tests

Pietrantuono, Giamattei & Russo, Reasoning Beyond Prediction: From Data-Driven to Causal Software Engineering, CACM (to appear in Dec 2026) – https://arxiv...

REFERENCES

Sources & further reading – TO INTEGRATE

Framework & roadmap

Pietrantuono, Giamattei, Russo. Reasoning Beyond Prediction: From Data-Driven to Causal Software Engineering to appear in CACM, 2026, https://arxiv.org/abs/2606.27960

- Causal Software Engineering: A Vision and Roadmap. arXiv:2605.02454.
- Giamattei et al. Causal reasoning in SQA: a systematic review. IST 178 (2025) 107599.

Testing

- Giamattei et al. Causality-driven Testing of Autonomous Driving Systems. ACM TOSEM 33(3), 2024.
- Giamattei et al. Identifying Performance Issues in Microservices through Causal Reasoning. AST 2024.
- Mascia et al. Microservices performance testing (causality-guided / LLM), 2025.
- Clark et al. Metamorphic Testing with Causal Graphs. ICST 2023.
- Foster et al. Causal inference for systems with hidden/interacting variables. EASE 2025.

Fault localization & RCA

- Johnson, Brun, Meliou. Causal testing: understanding defects' root causes. ICSE 2020.
- Wu, Tordsson, Elmroth, Kao. Causal Inference Techniques for Microservice Performance. 2021.
- PyWhy / DoWhy - GCM root-cause analysis example (pywhy.org).

Requirements engineering

- Heyn et al. Causal models as a Requirements Engineering activity (arc-fault study).
- Ahmad et al. RE for AI systems: a systematic mapping. IST 158 (2023) 107176.
- D'Amour et al. Underspecification in modern ML. JMLR 23 (2022).
- Mitchell et al. Algorithmic fairness: choices, assumptions, definitions. 2021.

Validity & caveats

- Hulse, Eisty, Menzies. Shaky structures: causal graphs in software analytics. EMSE 30 (2025).